06_maintenance

August 18, 2026

1 NB6 — Table Maintenance: the 4 jobs that are not optional

Stack: `deltalake` + `pyiceberg`. Maps to slide §6 (Storage Optimization → *Table Maintenance: 4 Job Bắt Buộc*) + §12 FinOps + deliverable bullet 6.

> **The small-file problem is the single most common production failure mode of a lakehouse** — more common than every other cause combined. It is not caused by bad code. It is caused by *normal* streaming ingestion plus the absence of a cron job.

The four mandatory jobs, per the slide:

#	Job	If you skip it	Delta	Iceberg
1	Compaction	Small files → slow queries, non-linear cost	`OPTIMIZE`	`rewrite_data_files`
2	Clustering / sort	Loose stats → no file skipping	`ZORDER` / Liquid	`sort` / `zorder`
3	Snapshot expiry	Metadata bloat, slow listing, storage garbage	`VACUUM RETAIN`	`expire_snapshots`
4	Orphan removal	You pay for invisible garbage from failed jobs	`VACUUM`	`remove_orphan_files`

You will run all four and **measure** each one.

[1]:
```
import _setup  # noqa: F401

import datetime as dtm
import glob
import os
import time
from pathlib import Path

import pyarrow as pa
import pyarrow.parquet as pq
from deltalake import DeltaTable, write_deltalake
```

```
from lakehouse import catalog, count_files, du, human, namespace, path, reset,␣
 ↪reset_catalog

TABLE = path("scratch", "maint_events")
reset(TABLE)
```

1.1 0. Manufacture the problem: 200 micro-batches

This is what a Kafka→lakehouse job with a 5-second trigger does to you overnight. Each commit is small and perfectly correct. The *accumulation* is the bug.

[2]:
```
import random

N_BATCHES, ROWS_PER_BATCH = 200, 500
USERS = 20_000
random.seed(42)
ALPHABET = "abcdefghijklmnopqrstuvwxyz0123456789"

t0 = time.perf_counter()
for b in range(N_BATCHES):
    batch = pa.table({
        "event_id":   [b * ROWS_PER_BATCH + i for i in range(ROWS_PER_BATCH)],
        "user_id":    [(b * 7919 + i * 31) % USERS for i in␣
 ↪range(ROWS_PER_BATCH)],
        "ts":         [dtm.datetime(2026, 8, 1 + b % 10, b % 24, i % 60) for i␣
 ↪in range(ROWS_PER_BATCH)],
        "latency_ms": [200 + (i * 13) % 2500 for i in range(ROWS_PER_BATCH)],
        # A prompt/response blob. Without a wide column the rows are so tiny
        # that everything compacts into one file and there is nothing to skip.
        "payload":    ["".join(random.choices(ALPHABET, k=80)) for _ in␣
 ↪range(ROWS_PER_BATCH)],
    })
    write_deltalake(TABLE, batch, mode="append" if b else "overwrite")
ingest_s = time.perf_counter() - t0


def snapshot_metrics(label: str) -> dict:
    """The four numbers that actually matter. Deterministic - not wall-clock."""
    dt = DeltaTable(TABLE)
    log = Path(TABLE) / "_delta_log"
    m = {
        "data files":     len(dt.file_uris()),
        "data bytes":     du(TABLE) - du(log),
        "log bytes":      du(log),
        "log entries":    len(list(log.glob("*.json"))),
        "rows":           dt.count(),
    }
```

```
    print(f"{label:24s} files={m['data files']:>4}  data={human(m['data␣
 ↪bytes']):>9}  "
          f"log={human(m['log bytes']):>9} ({m['log entries']} json) ␣
 ↪rows={m['rows']:,}")
    return m


print(f"Ingested {N_BATCHES * ROWS_PER_BATCH:,} rows in {N_BATCHES} commits␣
 ↪({ingest_s:.1f}s)\n")
base = snapshot_metrics("BASELINE (the mess)")
print(f"\nAverage file size: {human(base['data bytes'] / base['data files'])}"
      "   ← production target is 128-512 MB")
```

```
Ingested 100,000 rows in 200 commits (11.1s)

BASELINE (the mess)      files= 200  data=  10.1 MB  log= 382.6 KB (200 json)
rows=100,000

Average file size: 51.5 KB   ← production target is 128-512 MB
```

1.1.1 Why small files cost non-linearly

Object storage charges per **request**, not just per byte, and every file costs a `GET` (plus metadata to plan over). 200 files is invisible; 200 million files is a platform outage.

[3]:
```
GET_PER_1K = 0.0004     # S3 GET list price, USD per 1,000 requests
QUERIES_PER_DAY = 50_000
gets_now = base["data files"] * QUERIES_PER_DAY
print(f"Full-scan GETs/day at {base['data files']} files: {gets_now:,}"
      f"  →  ${gets_now / 1000 * GET_PER_1K:,.2f}/day just in requests")
print(f"Same data in 4 compacted files:      {4 * QUERIES_PER_DAY:,}"
      f"  →  ${4 * QUERIES_PER_DAY / 1000 * GET_PER_1K:,.2f}/day")
```

```
Full-scan GETs/day at 200 files: 10,000,000  →  $4.00/day just in requests
Same data in 4 compacted files:      200,000  →  $0.08/day
```

1.2 Job 1 — Compaction

`OPTIMIZE` rewrites many small files into few large ones. It is a *new commit*: the old files are not deleted, they are **tombstoned** (that matters for Job 3).

[4]:
```
# Lab scale: 1 MB target. Production targets 128-512 MB - the *ratio* of
# before:after is the lesson, not the absolute size.
TARGET_SIZE = 1024 * 1024

dt = DeltaTable(TABLE)
metrics = dt.optimize.compact(target_size=TARGET_SIZE)
after_compact = snapshot_metrics("AFTER compaction")
```

```
print(f"\nfilesAdded={metrics['numFilesAdded']} ␣
  ↪filesRemoved={metrics['numFilesRemoved']}")
print(f"File reduction: {base['data files']} → {after_compact['data files']} "
      f"({base['data files'] / max(after_compact['data files'], 1):.0f}×␣
  ↪fewer)")
print("Note: data bytes went UP slightly - compaction WRITES new files before")
print("the old ones are reclaimed. You pay twice, briefly. Budget for it.")
```

```
AFTER compaction         files=  11  data=  16.1 MB  log= 431.3 KB (201 json)
rows=100,000

filesAdded=11  filesRemoved=200
File reduction: 200 → 11 (18× fewer)
Note: data bytes went UP slightly - compaction WRITES new files before
the old ones are reclaimed. You pay twice, briefly. Budget for it.
```

1.3 Job 2 — Clustering, measured by *stats quality* (not by stopwatch)

NB2 timed Z-ORDER. Timing is noisy on a laptop, so here we measure the thing timing is a proxy for: **how many files could possibly contain the row you want**, according to per-file min/max stats in the log.

That count is what the engine skips on. It is deterministic.

[5]:
```
TARGET_USER = 12_345


def files_touched(target: int = TARGET_USER) -> int:
    """Files whose [min,max] user_id range could contain `target`."""
    # delta-rs 1.x returns an arro3 Table; pa.table() adopts it via the
    # Arrow C stream interface (zero copy, no pandas needed).
    aa = pa.table(DeltaTable(TABLE).get_add_actions(flatten=True)).to_pylist()
    return sum(1 for f in aa if f["min.user_id"] <= target <= f["max.user_id"])


before_cluster = files_touched()
DeltaTable(TABLE).optimize.z_order(["user_id"], target_size=TARGET_SIZE)
after_cluster = files_touched()
total_files = len(DeltaTable(TABLE).file_uris())

print(f"Point query user_id={TARGET_USER}")
print(f"  before clustering: must open {before_cluster}/{after_compact['data␣
  ↪files']} files")
print(f"  after  clustering: must open {after_cluster}/{total_files} files")
print(f"  → skip rate: {(1 - after_cluster / max(total_files, 1)) * 100:.0f}%␣
  ↪of files never touched")
print("\nUnclustered data has overlapping min/max ranges, so stats prove␣
  ↪nothing")
```

```
print("and the engine must read everything. Clustering is what makes stats␣
  ↪USEFUL.")
```

```
Point query user_id=12345
  before clustering: must open 11/11 files
  after  clustering: must open 1/10 files
  → skip rate: 90% of files never touched

Unclustered data has overlapping min/max ranges, so stats prove nothing
and the engine must read everything. Clustering is what makes stats USEFUL.
```

1.4 Job 3 — Snapshot / version expiry

Every commit so far is still fully readable — that's time travel, and it is not free. Those tombstoned files are still on disk and still billed.

`retention_hours=0` destroys your ability to time-travel and can break readers mid-query. Production uses 168h (7 days). We force it here only to make the reclaim visible inside a lab.

[6]:
```
dt = DeltaTable(TABLE)
doomed = dt.vacuum(retention_hours=0, dry_run=True,␣
  ↪enforce_retention_duration=False)
print(f"VACUUM would reclaim {len(doomed)} tombstoned files "
      f"({human(sum(du(f) for f in doomed))})")

before_vacuum = du(TABLE)
dt.vacuum(retention_hours=0, dry_run=False, enforce_retention_duration=False)
after_vacuum = snapshot_metrics("AFTER vacuum")
print(f"\nReclaimed: {human(before_vacuum - du(TABLE))}")
print(f"Time travel to v0 is now GONE - that is the trade you just made.")
```

```
VACUUM would reclaim 211 tombstoned files (0 B)
AFTER vacuum             files=  10  data=   6.2 MB  log= 441.9 KB (204 json)
rows=100,000

Reclaimed: 16.1 MB
Time travel to v0 is now GONE - that is the trade you just made.
```

1.5 Job 4 — Orphan files: the garbage nobody can see

An orphan is a file written by a job that **crashed before committing**. It is on disk, you are billed for it, and *no metadata references it* — so it is invisible to `history()`, to `files()`, and to your dashboards.

We simulate three crashed writers:

[7]:
```
for i in range(3):
    orphan = Path(TABLE) / f"part-9999{i}-crashed-writer-c000.snappy.parquet"
```

```
    pq.write_table(pa.table({"event_id": list(range(1000)), "user_id": [0] *␣
 ↪1000,
                             "ts": [dtm.datetime(2026, 8, 1)] * 1000,
                             "latency_ms": [1] * 1000,
                             "payload": ["x" * 80] * 1000}), orphan)
    old = time.time() - 30 * 86400          # 30 days old - well past any␣
 ↪retention
    os.utime(orphan, (old, old))

dt = DeltaTable(TABLE)
print(f"Rows reported by the table: {dt.count():,}   (orphans are invisible)")
print(f"Parquet files on disk:      {count_files(TABLE)}")
print(f"Parquet files in the log:   {len(dt.file_uris())}")
print(f"→ {count_files(TABLE) - len(dt.file_uris())} files you pay for and␣
 ↪cannot see")
```

```
Rows reported by the table: 100,000   (orphans are invisible)


Parquet files on disk:      15
Parquet files in the log:   10
→ 5 files you pay for and cannot see
```

1.5.1 Measured finding: `VACUUM` alone does not catch these

Run vacuum again on a table whose orphans are 30 days old:

[8]:
```
still = DeltaTable(TABLE).vacuum(retention_hours=0, dry_run=True,␣
 ↪enforce_retention_duration=False)
print(f"VACUUM dry-run now finds: {len(still)} files")
print(f"Orphans still on disk:    {count_files(TABLE) - len(DeltaTable(TABLE).
 ↪file_uris())}")
print("""
`deltalake` (the Rust/Python implementation used here) reclaims files the
transaction log has TOMBSTONED. A file that was never committed was never
tombstoned, so the log has no idea it exists.

Spark's VACUUM additionally lists the table directory, which is why the slide
lists VACUUM under orphan removal - but you should never *assume* your engine
does the directory pass. Verify it, or run the diff yourself:
""")
```

```
VACUUM dry-run now finds: 211 files


Orphans still on disk:    5

`deltalake` (the Rust/Python implementation used here) reclaims files the
transaction log has TOMBSTONED. A file that was never committed was never
```

tombstoned, so the log has no idea it exists.

Spark's VACUUM additionally lists the table directory, which is why the slide lists VACUUM under orphan removal - but you should never *assume* your engine does the directory pass. Verify it, or run the diff yourself:

1.5.2 The orphan-removal algorithm (this is all `remove_orphan_files` does)

Set difference: *files on disk − files referenced by live metadata*, with an age guard so you never delete a file an in-flight writer is still using.

[9]:
```
def find_orphans(table_path: str, min_age_hours: int = 24) -> list[str]:
    """Files on disk that no live snapshot references, older than the guard."""
    referenced = {os.path.realpath(u.replace("file://", ""))
                  for u in DeltaTable(table_path).file_uris()}
    cutoff = time.time() - min_age_hours * 3600
    orphans = []
    for f in Path(table_path).rglob("*.parquet"):
        if "_delta_log" in f.parts:
            continue
        if os.path.realpath(f) not in referenced and f.stat().st_mtime < cutoff:
            orphans.append(str(f))
    return orphans


found = find_orphans(TABLE)
reclaimed = sum(du(f) for f in found)
print(f"Orphans found: {len(found)}  ({human(reclaimed)})")
for f in found:
    print(f"  {os.path.basename(f)}")
    os.remove(f)

print(f"\nAfter removal - on disk: {count_files(TABLE)}, in log:␣
 ↪{len(DeltaTable(TABLE).file_uris())}")
print("\n  The age guard is not optional. Without it you will delete files that␣
 ↪a")
print("   concurrent writer has written but not yet committed, and corrupt the␣
 ↪table.")
```

```
Orphans found: 3  (21.2 KB)
  part-99990-crashed-writer-c000.snappy.parquet
  part-99991-crashed-writer-c000.snappy.parquet
  part-99992-crashed-writer-c000.snappy.parquet

After removal - on disk: 12, in log: 10

  The age guard is not optional. Without it you will delete files that a
```

```
concurrent writer has written but not yet committed, and corrupt the table.
```

1.6 Job 5 — Manifest / log rewrite (the streaming tax)

200 commits = 200 JSON log entries. A cold reader replays **all of them** to learn the current state. A checkpoint collapses that into one Parquet file.

[10]:
```
log_dir = Path(TABLE) / "_delta_log"
json_before = len(list(log_dir.glob("*.json")))

DeltaTable(TABLE).create_checkpoint()

ckpt = list(log_dir.glob("*.checkpoint.parquet"))
print(f"JSON log entries a cold reader would replay: {json_before}")
print(f"Checkpoint written: {ckpt[0].name if ckpt else 'NONE'}")
print(f"_last_checkpoint present: {(log_dir / '_last_checkpoint').exists()}")
print("\nA reader now loads 1 checkpoint + the few JSONs after it, not all 200.
 ↪")
print("For CDC/streaming tables this is the difference between a 200 ms and a")
print("20 s cold start - and it is why the slide calls it the 5th job.")
```

```
JSON log entries a cold reader would replay: 204
Checkpoint written: 00000000000000000099.checkpoint.parquet
_last_checkpoint present: True

A reader now loads 1 checkpoint + the few JSONs after it, not all 200.
For CDC/streaming tables this is the difference between a 200 ms and a
20 s cold start - and it is why the slide calls it the 5th job.
```

1.7 The same four jobs on Iceberg

Different spelling, identical obligations. `expire_snapshots` is the one pyiceberg exposes natively today; compaction (`rewrite_data_files`) and `remove_orphan_files` are Spark/Java procedures — so on the Python path you run the diff yourself, exactly as above.

[11]:
```
CAT = "nb6"          # own catalog dir - see scripts/lakehouse.py:_catalog_dir
reset_catalog(CAT)
cat = catalog(CAT)
ns = namespace(cat, "lake")

ICE_SCHEMA = pa.schema([
    pa.field("event_id", pa.int64(), nullable=False),
    pa.field("user_id", pa.int64()),
])
ice = cat.create_table(f"{ns}.maint", schema=ICE_SCHEMA)
for b in range(20):
    ice.append(pa.table({"event_id": list(range(b * 100, (b + 1) * 100)),
                         "user_id": [(b * 31 + i) % 5000 for i in range(100)]},␣
 ↪schema=ICE_SCHEMA))
```

```
ice = cat.load_table(f"{ns}.maint")

ice_loc = ice.location().replace("file://", "")
ice_meta = Path(ice_loc) / "metadata"


def ice_metrics(label: str) -> dict:
    t = cat.load_table(f"{ns}.maint")
    m = {
        "snapshots":      len(t.snapshots()),
        "manifest avro":  len(list(ice_meta.glob("*.avro"))),
        "metadata json":  len(list(ice_meta.glob("*.json"))),
        "metadata bytes": du(str(ice_meta)),
    }
    print(f"{label:14s} snapshots={m['snapshots']:>3}  avro={m['manifest avro']:
 ↪>3}  "
          f"json={m['metadata json']:>3}  metadata={human(m['metadata␣
 ↪bytes'])}")
    return m


ice_before = ice_metrics("before expiry")
```

```
before expiry  snapshots= 20  avro= 40  json= 21  metadata=325.2 KB
```

1.7.1 `expire_snapshots` — keep the newest, drop the rest

Watch the avro count, not just the snapshot count.

[12]:
```
KEEP_LAST = 3
doomed_ids = [s.snapshot_id for s in ice.snapshots()[:-KEEP_LAST]]
ice.maintenance.expire_snapshots().by_ids(doomed_ids).commit()
ice = cat.load_table(f"{ns}.maint")

ice_after = ice_metrics("after expiry")
print(f"\nRows still intact: {ice.scan().to_arrow().num_rows:,}")
```

```
after expiry   snapshots=  3  avro= 40  json= 22  metadata=332.3 KB

Rows still intact: 2,000
```

1.7.2 Measured finding: expiry is a metadata-only operation

Snapshots dropped from 20 to 3 — but *not one avro file was deleted*, and metadata on disk actually grew (expiry writes a new `metadata.json`).

This is not a bug. Expiry's job is to make files **unreferenced**. Deleting them is a *different* job — Job 4. In Java/Spark Iceberg the two are usually chained; on the Python path you must chain them yourself, or the storage never shrinks.

[13]:
```
print(f"snapshots: {ice_before['snapshots']} → {ice_after['snapshots']}  "
      f"({ice_before['snapshots'] - ice_after['snapshots']} expired)")
print(f"avro files on disk: {ice_before['manifest avro']} →␣
 ↪{ice_after['manifest avro']}  "
      f"(deleted: {ice_before['manifest avro'] - ice_after['manifest avro']})")
print(f"metadata bytes: {human(ice_before['metadata bytes'])} →␣
 ↪{human(ice_after['metadata bytes'])}")
print("\nExpiry alone reclaimed NOTHING. Now find what it stranded:")
```

```
snapshots: 20 → 3  (17 expired)
avro files on disk: 40 → 40  (deleted: 0)
metadata bytes: 325.2 KB → 332.3 KB

Expiry alone reclaimed NOTHING. Now find what it stranded:
```

[14]:
```
def find_iceberg_orphans(table) -> list[Path]:
    """Manifest lists on disk that no live snapshot points at.

    Iceberg names manifest lists `snap-<snapshot-id>-...avro`, and each live
    snapshot records its own in `manifest_list`. The set difference is garbage
    - the same diff `remove_orphan_files` performs.
    """
    live = {Path(s.manifest_list).name for s in table.snapshots()}
    return [f for f in ice_meta.glob("snap-*.avro") if f.name not in live]


stranded = find_iceberg_orphans(ice)
print(f"Stranded manifest lists: {len(stranded)}  ({human(sum(du(f) for f in␣
 ↪stranded))})")
for f in stranded[:3]:
    print(f"  {f.name}")
if len(stranded) > 3:
    print(f"  ... and {len(stranded) - 3} more")

reclaimed_ice = sum(du(f) for f in stranded)
for f in stranded:
    f.unlink()

ice_final = ice_metrics("after sweep")
print(f"\nReclaimed by chaining expiry → orphan removal:␣
 ↪{human(reclaimed_ice)}")
print(f"Rows still intact: {cat.load_table(f'{ns}.maint').scan().to_arrow().
 ↪num_rows:,}")
print("\n→ Job 3 and Job 4 are a PAIR. Running expiry without a sweep is why")
print("  teams report 'we expire snapshots but the S3 bill never drops'.")
```

```
Stranded manifest lists: 17  (36.5 KB)
  snap-2849133556814387238-0-2a300d73-0c59-43ab-9612-a3af4b8ca88b.avro
```

```
  snap-3400825954060120835-0-509b315e-7ca9-48be-96a5-7bf17dc07915.avro
  snap-3473544185682471274-0-f39f982e-a786-4b10-a863-a69bcb774e4d.avro
  … and 14 more
after sweep    snapshots=  3  avro= 23  json= 22  metadata=295.9 KB

Reclaimed by chaining expiry → orphan removal: 36.5 KB
Rows still intact: 2,000

→ Job 3 and Job 4 are a PAIR. Running expiry without a sweep is why
  teams report 'we expire snapshots but the S3 bill never drops'.
```

1.8 Who runs these jobs? Self-managed vs managed

"Fully managed" free. Managed compaction meters **per GB processed** *and* **per 1,000 objects** — a table with pathological small files is exactly the table that is most expensive to have auto-compacted.

[15]:
```
TABLE_GB, FILES, COMPACTIONS_PER_MONTH = 500, 2_000_000, 30
PRICE_GB, PRICE_PER_1K_OBJ = 0.05, 0.004

gb_cost = TABLE_GB * PRICE_GB * COMPACTIONS_PER_MONTH
obj_cost = FILES / 1000 * PRICE_PER_1K_OBJ * COMPACTIONS_PER_MONTH
print(f"Managed compaction, {TABLE_GB} GB / {FILES:,} files, daily:")
print(f"  per-GB component:     ${gb_cost:,.0f}/mo")
print(f"  per-object component: ${obj_cost:,.0f}/mo")
print(f"  TOTAL:                ${gb_cost + obj_cost:,.0f}/mo")
print(f"\nThe object component is {obj_cost / (gb_cost + obj_cost) * 100:.0f}%␣
  ↪of the bill -")
print("it is driven by FILE COUNT, not data volume. Fixing your writer's")
print("trigger interval is cheaper than paying someone to clean up after it.")
```

```
Managed compaction, 500 GB / 2,000,000 files, daily:
  per-GB component:     $750/mo
  per-object component: $240/mo
  TOTAL:                $990/mo

The object component is 24% of the bill -
it is driven by FILE COUNT, not data volume. Fixing your writer's
trigger interval is cheaper than paying someone to clean up after it.
```

1.9 NB6 pass criteria

Check	Target
Compaction	10× fewer files
Clustering	50% of files skippable for a point query
Snapshot expiry	Delta reclaims bytes; Iceberg drops to 3 snapshots

Check	Target
Orphan removal	3 planted Delta orphans + all stranded Iceberg manifest lists swept
Log checkpoint	`*.checkpoint.parquet` + `_last_checkpoint` exist

[16]:
```
checks = {
    "compaction  10x fewer files": base["data files"] /␣
 ↪max(after_compact["data files"], 1) >= 10,
    "clustering skips  50% files": (1 - after_cluster / max(total_files, 1))␣
 ↪>= 0.5,
    "vacuum reclaimed bytes":       before_vacuum > du(TABLE),
    "3 delta orphans removed":      len(found) == 3,
    "no delta orphans remain":      find_orphans(TABLE) == [],
    "checkpoint written":           bool(ckpt) and (log_dir /␣
 ↪"_last_checkpoint").exists(),
    "iceberg expired to 3 snaps":   len(ice.snapshots()) == KEEP_LAST,
    "iceberg stranded files swept": len(stranded) > 0 and␣
 ↪find_iceberg_orphans(ice) == [],
    "iceberg data intact":          cat.load_table(f"{ns}.maint").scan().
 ↪to_arrow().num_rows == 2000,
}
for k, v in checks.items():
    print(f"  [{'PASS' if v else 'FAIL'}] {k}")
assert all(checks.values()), "NB6 incomplete - see FAIL rows above"
print("\nNB6 complete.")
```

```
  [PASS] compaction  10x fewer files
  [PASS] clustering skips  50% files
  [PASS] vacuum reclaimed bytes
  [PASS] 3 delta orphans removed
  [PASS] no delta orphans remain
  [PASS] checkpoint written
  [PASS] iceberg expired to 3 snaps
  [PASS] iceberg stranded files swept
  [PASS] iceberg data intact

NB6 complete.
```